

Design-Weighted Latent Class Analysis of Survey Data: An Adaptable Analysis Script

Institutional trust in Mexico: LAPOP AmericasBarometer 2023

Kevin

2026-07-15

1 Purpose and method

This document estimates a population-level latent class typology from complex-survey items and carries the sampling design through every result: enumeration, parameter intervals, class assignment, and demographic profiling all rest on the same design. Estimation maximizes the design-weighted pseudo-log-likelihood by EM; all uncertainty comes from stratified jackknife (JKn) replicate weights, because a pseudo-likelihood has no usable information matrix. Everything dataset-specific lives in the `pd` block and the configuration; the machinery lives in `survey_lca_source.R` and never changes.

The decisions taken, each argued in the section that uses it: weights enter every EM step, so estimates describe the population, not the achieved sample; information criteria rescale the log-likelihood to the sum-to-n scale, otherwise BIC leans toward too many classes; the bootstrapped LRT is omitted because its independent-observations premise fails under a design-weighted pseudo-likelihood on a clustered sample; the analysis uses complete cases only, and K is the analyst's decision (`cfg$K_force`), made by iterating over the enumeration and the diagnostics rather than auto-selected; demographic profiling is design-based domain estimation on soft posteriors, not a three-step regression; class labels follow one rule (`outputs/llm_labels.csv` is used when it exists, otherwise the LLM drafts it once and writes it); and membership is exported to a labelled SPSS file for analysts once K is set and the labels exist.

2 Setup

```
library(tidyverse)      # data manipulation and plotting
library(matrixStats)    # numerically stable row-wise log-sum-exp and cumulative sums
library(here)
library(survey)         # complex-design objects, replicate weights, svyglm
library(poLCA)          # unweighted benchmark latent class model
library(clue)           # solve_LSAP: optimal label alignment across fits
library(furrr)          # future_map_*: parallel model fits (enumeration, screen, poLCA)
```

```

# All reusable machinery lives in one annotated source file: the plotting and
# data-prep helpers, the weighted EM engine and label alignment, the LLM
# class-labeling functions. Keep
# survey_lca_source.R in the same folder as
# this document; the render fails if it is not on the path. Sourcing it here, after
# the libraries, means every function is available to the analysis chunks below.
source("survey_lca_source.R")

`%|||%` <- rlang::`%|||%` # null-coalescing helper (also defined in the source file)
# Reproducibility is controlled by cfg$seed (set in the configuration block): every
# random step below (the EM random starts and the polCA starts) seeds
# itself from it, so the classes and their labels are identical across runs.

# polCA attaches MASS, whose select() and some matrixStats verbs (count) mask
# their tidyverse equivalents. We namespace-qualify dplyr::select(), dplyr::count(),
# and dplyr::distinct() throughout. This is a deliberate, defensive choice.

```

3 Reading and processing the survey data

Data cleaning is front-loaded and transparent: the two chunks below are ordinary dplyr you own and edit per dataset, and everything after them is dense machinery that never changes. The first chunk reads the raw SPSS file, recodes nonresponse to NA, recodes the demographics to character, flags the analysis cases (complete on the items and the demographics; cases missing every item are simply among the dropped), and strips the “don’t know”/“refusal” value labels from the item attributes so that every label left on an item is a real response. The original processed frame is preserved for the membership export at the end.

```

raw_survey_dat <- haven::read_sav(
  here("LCA/data/MEX_2023_LAPOP_AmericasBarometer_v1.0_w.sav"))

items      <- c(justice_system = "b10a", electoral_tribunal = "b11",
               armed_forces = "b12", legislature = "b13",
               public_ministry = "b15", police = "b18", auditor = "b19",
               political_parties = "b21", president = "b21a",
               supreme_court = "b31", municipality = "b32", media = "b37",
               elections = "b47a")
design_vars <- c(id = "idnum", strata = "strata", psu = "upm", weight = "wt")
na_codes   <- c(888888, 988888, 999999) # DK / NR / N-A in this file

process_survey_dat <- raw_survey_dat |>
  dplyr::select(all_of(unname(design_vars)), all_of(unname(items)),
               q1tc_r, q2, edre, ur, ocup4a, pn4, m1) |>
  rename(!!!items) |>
  # items and design variables stay numeric; nonresponse codes to NA
  mutate(across(all_of(names(items)), ~ replace(as.numeric(.x), .x %in% na_codes, NA)),

```

```

    across(all_of(unname(design_vars)), as.numeric),
    across(c(pn4, m1, q2), ~ replace(as.numeric(.x), .x %in% na_codes, NA))) |>
# demographics recoded through their label text (as_character), kept character
mutate(
  age_cat = as.character(cut(q2, breaks = c(17, 29, 44, 59, Inf),
                                labels = c("18-29", "30-44", "45-59", "60+"))),
  male    = ifelse(sjlabelled::as_character(q1tc_r) == "Mujer/femenino",
                    "Female", "Male"),
  Edu      = case_when(
    sjlabelled::as_character(edre) == "Ninguna" ~ "none",
    str_detect(sjlabelled::as_character(edre), "^Primaria") ~ "primary",
    str_detect(sjlabelled::as_character(edre), "^Secundaria") ~ "secondary",
    str_detect(sjlabelled::as_character(edre), "^Universitaria") ~ "higher_ed",
    TRUE ~ NA_character_),
  Urban    = ifelse(sjlabelled::as_character(ur) == "Urbano", "urban", "rural"),
  Employment = case_when(
    str_detect(sjlabelled::as_character(ocup4a),
               "^Trabajando|pero tiene trabajo") ~ "Employed",
    str_detect(sjlabelled::as_character(ocup4a), "quehaceres") ~ "house_keeper",
    str_detect(sjlabelled::as_character(ocup4a), "estudiante") ~ "Student",
    str_detect(sjlabelled::as_character(ocup4a), "jubilado|pensionado") ~ "Retired",
    str_detect(sjlabelled::as_character(ocup4a), "buscando") ~ "Unemployed",
    TRUE ~ NA_character_),
  satis_demo = c("Very satisfied", "Satisfied", "Dissatisfied",
                 "Very dissatisfied")[pn4],
  prez_rating = c("Very good", "Good", "Neither good nor bad", "Bad",
                  "Very bad")[m1]
) |>
dplyr::select(-q1tc_r, -q2, -edre, -ur, -ocup4a, -pn4, -m1) |>
# strip nonresponse value labels so every label left on an item is a response
mutate(across(all_of(names(items)),
              ~ sjlabelled::remove_labels(.x, labels = na_codes))) |>
# the analysis flag: complete on items AND demographics (complete cases only)
mutate(keep = stats::complete.cases(
  across(c(all_of(names(items)), age_cat, male, Edu, Urban, Employment,
           satis_demo, prez_rating))))

survey_dat_original <- process_survey_dat          # preserved for the export
survey_dat          <- process_survey_dat |> dplyr::filter(keep)
items               <- names(items)

```

The second chunk documents what the model (and the class labeler) will see: the dictionary pairs each item with its question text and its response labels as text, and the single stacked plot shows the retained cases on the substantive scale.

```

dictionary <- tibble(
  item      = items,

```

```
question = map_chr(items, ~ sjlabelled::get_label(survey_dat[[.x]]) %||% .x),
responses = map_chr(items, function(it) {
  x <- survey_dat[[it]]
  v <- sort(unique(x[!is.na(x)]))
  lk <- set_names(as.character(sjlabelled::get_labels(x)),
    sjlabelled::get_values(x))
  paste(coalesce(lk[as.character(v)], as.character(v)), collapse = " | ")
}))
knitr::kable(dictionary,
  caption = "Item dictionary: the question text and response labels exactly as the c
```

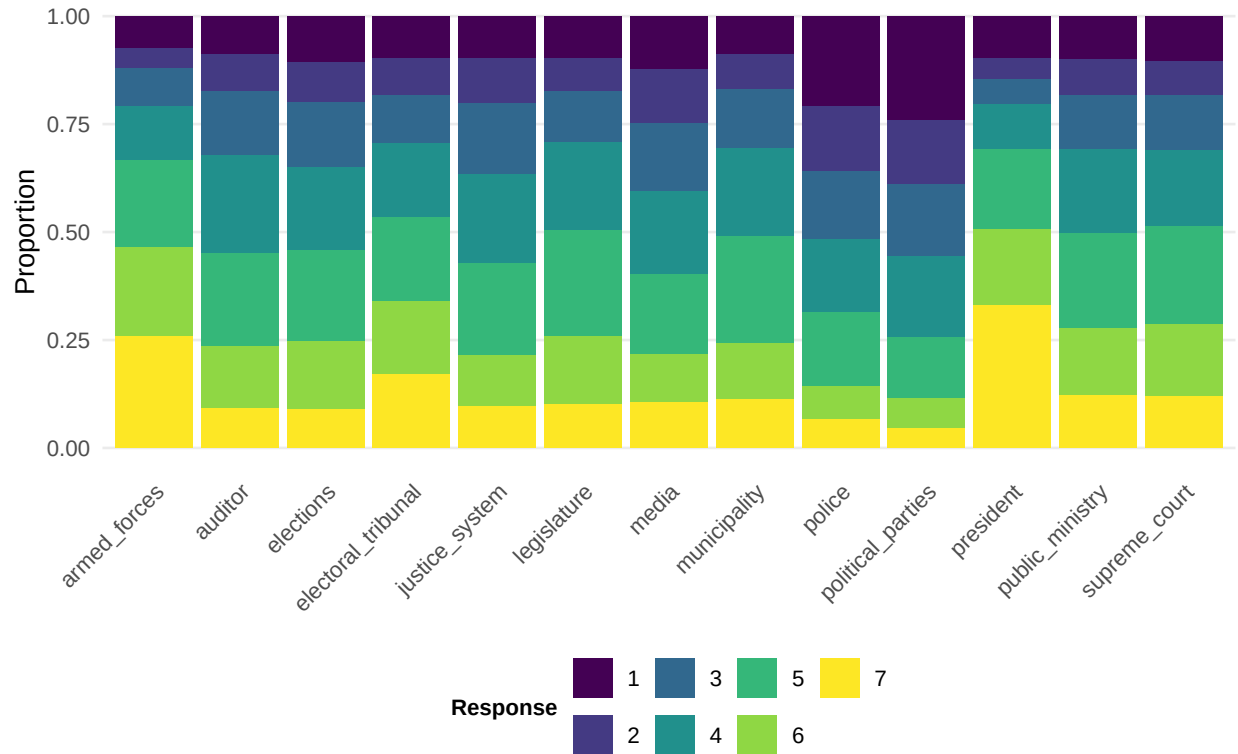
Table 1: Item dictionary: the question text and response labels exactly as the class labeler will read them.

item	question	responses
justice_system	justice_system	1 2 3 4 5 6 7
electoral_tribunal	electoral_tribunal	1 2 3 4 5 6 7
armed_forces	armed_forces	1 2 3 4 5 6 7
legislature	legislature	1 2 3 4 5 6 7
public_ministry	public_ministry	1 2 3 4 5 6 7
police	police	1 2 3 4 5 6 7
auditor	auditor	1 2 3 4 5 6 7
political_parties	political_parties	1 2 3 4 5 6 7
president	president	1 2 3 4 5 6 7
supreme_court	supreme_court	1 2 3 4 5 6 7
municipality	municipality	1 2 3 4 5 6 7
media	media	1 2 3 4 5 6 7
elections	elections	1 2 3 4 5 6 7

```
cat(nrow(survey_dat), "of", nrow(survey_dat_original),
  "cases are complete on the items and demographics and enter the analysis.\n")
```

1430 of 1622 cases are complete on the items and demographics and enter the analysis.

```
print(plot_item_stack(survey_dat, items, "Retained cases, substantive scale"))
```



4 Configuration

`cfg` is the analysis contract. `K_force` is the analyst's decision, made by iterating: leave it `NULL`, render, read the enumeration table (BIC, entropy, inclusion probabilities), then set a candidate `K` and re-render to see that model's item discrimination, bivariate residuals, and profiles; adjust until satisfied. Nothing downstream of enumeration runs while `K_force` is `NULL`. Class labels follow one rule: when `K_force` is set, `outputs/llm_labels.csv` is used if it exists, and otherwise the LLM drafts it once (one call per class) and writes it; edit that file to take over naming. API keys are never handled in code: `ellmer` reads `OPENAI_API_KEY` from `.Renviron`, which is where they live at home and at work.

```
cfg <- list(
  # --- estimation -----
  items      = items,
  strata     = "strata", psu = "upm", weight = "wt",
  K_range    = 2:8,          # candidates shown in the enumeration table
  K_force    = 5,           # THE analyst decision; set after reviewing diagnostics
  n_starts   = 20, seed = 2026, parallel = TRUE, workers = NULL,
  aux        = c("age_cat", "male", "Edu", "Urban", "Employment"),
  na_codes   = NULL,        # items arrive clean from the processing chunk

  # --- outputs -----
  out_dir    = here("LCA/outputs"),
```

```

# --- LLM class labeling -----
compass_base_url = "https://openrouter.ai/api/v1", # work: your compass endpoint
llm_model        = "google/gemma-4-31b-it",        # work: your model name
# Add this to your configuration setup from .rEnvironment
llm_key_env      = "OPENROUTER_API_KEY",
survey_context   = paste(
  "These items come from the 2023 AmericasBarometer survey of Mexico,",
  "conducted by the LAPOP Lab at Vanderbilt University. The AmericasBarometer",
  "is a comparative public opinion study of democratic attitudes and",
  "governance across the Americas, fielded to national probability samples",
  "of voting-age adults.",
  "\n\nThe battery analyzed here measures trust in national institutions:",
  "respondents rate, for each institution, how much they trust it on a",
  "seven-point scale anchored at 1 (not at all) and 7 (a lot). The latent",
  "classes summarize patterns of institutional trust across these items."),

data = survey_dat          # complete-case analysis frame from the chunks above
)
dir.create(cfg$out_dir, showWarnings = FALSE, recursive = TRUE)
init_parallel(cfg)
K_ready <- !is.null(cfg$K_force) # gates every section past enumeration

```

5 The model

Each respondent belongs to one of K unobserved classes, and within a class the items are independent (local independence). With π_k the class prevalences and ρ_{jck} the probability that class k gives response c to item j ,

$$P(\mathbf{y}_i) = \sum_{k=1}^K \pi_k \prod_{j=1}^J \rho_{j y_{ij} k}, \quad \sum_{c=1}^{C_j} \rho_{jck} = 1.$$

Items are nominal and may have different category counts. Under a complex design the unweighted likelihood targets the achieved sample, so estimation maximizes the design-weighted pseudo-log-likelihood $\ell_w(\theta) = \sum_i w_i \log(\sum_k \pi_k \prod_j \rho_{j y_{ij} k})$, the standard device for LCA under complex sampling (Patterson, Dayton, and Graubard 2002, *Journal of the American Statistical Association* 97(459):721-728); `poLCA` has no weights argument, which is why the engine is custom and `poLCA` serves as the unweighted benchmark. Local independence is the load-bearing assumption; the indicator screen and the bivariate residual test where it strains.

6 Preparing and validating the data

`prepare_items()` (source file) verifies every configured column exists, coerces the design columns to base types, and recodes each item to consecutive integers from 1 over its substantive categories

(the data arrive complete-case from the processing chunk, so nothing is split here). It also fixes the sum-to-n scale for the information criteria, documented at its definition. Only the per-item summary prints.

```
prep_fit      <- prepare_items(cfg)
dat_prepared  <- prep_fit$dat_prepared
cats          <- prep_fit$cats
w_vec         <- prep_fit$w_vec
scale_ic      <- prep_fit$scale_ic

knitr::kable(prep_fit$summary, align = "lrr",
             caption = "Per-item category counts and missing footprint after the NA recode.")
```

Table 2: Per-item category counts and missing footprint after the NA recode.

item	categories	n_missing
justice_system	7	0
electoral_tribunal	7	0
armed_forces	7	0
legislature	7	0
public_ministry	7	0
police	7	0
auditor	7	0
political_parties	7	0
president	7	0
supreme_court	7	0
municipality	7	0
media	7	0
elections	7	0

All 1430 prepared respondents are complete on the items and enter the fit.

7 The weighted EM estimator

The script estimates the measurement model with a weighted Expectation-Maximization (EM) algorithm. EM alternates two steps until the weighted log-likelihood stops improving.

The **E-step** computes each respondent's posterior probability of belonging to each class, given the current parameters:

$$p_{ik} = \frac{\pi_k \prod_j \rho_{j y_{ij} k}}{\sum_m \pi_m \prod_j \rho_{j y_{ij} m}}.$$

The **M-step** updates the parameters as *weighted* proportions, which is the only place the base weights enter:

$$\hat{\pi}_k = \frac{\sum_i w_i p_{ik}}{\sum_i w_i}, \quad \hat{\rho}_{jck} = \frac{\sum_i w_i p_{ik} \mathbf{1}(y_{ij} = c)}{\sum_i w_i p_{ik}}.$$

The $\hat{\rho}_{jck}$ denominator runs over respondents who answered item j ; because the analysis uses complete cases, that is every respondent, so the sum is the same $\sum_i w_i p_{ik}$ written above.

Why this section matters: this is the engine. The weighting in the M-step is the single mechanism that moves every estimate from describing the achieved sample to describing the population, so its correctness propagates to every prevalence, profile, and interval downstream. Because the objective being maximized is the weighted *pseudo*-log-likelihood rather than an ordinary likelihood, these are *pseudo*-maximum-likelihood estimates. They are consistent for the population parameters, but, as Section 10 explains, their standard errors cannot be read off the curvature of the objective in the usual way and must instead come from replication.

Two implementation choices make this robust and loop-free. First, the E-step is computed on the log scale and normalized with the **log-sum-exp** identity, which prevents numerical underflow when many small probabilities are multiplied together. Second, the iteration is expressed as a `purrr::reduce` *fold* over iteration indices rather than a `while` loop: each step takes the current parameter state and returns the updated one, and the fold stops once the weighted log-likelihood stops improving.

The estimator is implemented in `survey_lca_source.R`, where the code is annotated line by line: `em_run()` is the weighted EM fold, `make_inputs()` builds the integer item vectors and one-hot category matrices it consumes, `posterior_of()` is the standalone E-step (also usable to score new complete-case data), `align_to()` fixes label order across fits with the Hungarian algorithm, and `fit_lca()` runs many random starts and keeps the best weighted log-likelihood. Keeping the machinery there leaves this document focused on what the estimator means rather than on its bookkeeping.

8 Choosing the number of classes

The number of classes K is not known in advance and is chosen by comparing a sequence of models. Three quantities guide that choice, and because they are routinely misread, each is explained here in the terms you would use with an analyst.

BIC (Bayesian Information Criterion) scores a model by its fit, penalized for the number of parameters it uses:

$$\text{BIC} = -2\hat{\ell} + \text{df} \cdot \ln n, \quad \text{df} = (K - 1) + K \sum_j (C_j - 1).$$

The first term rewards fit (a higher log-likelihood $\hat{\ell}$ makes it smaller), and the second term charges a price for every additional parameter, a price that grows with the sample size n . **Lower BIC is better.** You read the column down the candidate values of K and look for the **minimum**; often the values fall steeply, reach a low point, and rise again, and that low point is the recommended number of classes. The interpretation is “the number of classes that explains the response patterns well without adding classes that only fit noise.” BIC is the primary tool for this decision in latent class analysis.

AIC (Akaike Information Criterion) has the same form with a smaller, fixed penalty (2 per parameter instead of $\ln n$):

$$\text{AIC} = -2\hat{\ell} + 2 \text{ df.}$$

It is reported for completeness, but because its penalty is lighter it tends to **favor too many classes**, so when AIC and BIC disagree, prefer BIC for a parsimonious, interpretable solution.

Relative entropy answers a different question. BIC and AIC ask “how many classes?”; entropy asks “once we have them, how cleanly are respondents sorted into them?”

$$E = 1 - \frac{-\sum_i \sum_k p_{ik} \ln p_{ik}}{n \ln K}.$$

It runs from 0 to 1. A value near 1 means most respondents have a posterior probability close to 1 for a single class, so the classification is sharp; a value near 0 means respondents are spread across classes with no clear home. A common rule of thumb treats values above about 0.8 as good separation. Entropy is **not** a measure of fit and must **not** be used to choose K on its own: a two-class model almost always has higher entropy than a five-class model simply because fewer classes are easier to separate. Read it alongside BIC as a description of how usable the resulting classification is, not as a selection criterion.

All enumeration here uses the **weighted** fit, so the chosen K describes the population, not the achieved sample. To keep the criteria honest, the weighted pseudo-log-likelihood is put on the sample-size scale before ℓ enters BIC and AIC: the weights are normalized to sum to n , so the fit term and the $\ln n$ penalty are measured in the same units. Without this the fit term would be inflated by the average weight and the criteria would lean toward too many classes. The script reports all three quantities across the candidate range and selects the K that minimizes the weighted BIC. One standard companion to BIC is deliberately absent: the **bootstrapped likelihood ratio test** (BLRT), the usual complement for comparing K against $K + 1$ classes because the ordinary likelihood ratio statistic is not chi-square distributed for that comparison. The BLRT’s parametric bootstrap simulates independent observations from the smaller model, and that premise does not carry over to a design-weighted pseudo-likelihood on a stratified, clustered sample, so applying it here would attach an invalid reference distribution to the decision. Enumeration therefore rests on the weighted BIC, read alongside entropy and the unweighted **poLCA** comparison in Section 12. *Why this section matters:* K is the one modeling choice with no default, and every later result is conditional on it, so a well-calibrated, defensible selection here is what keeps the prevalences and profiles from being artifacts of an over- or under-fit class structure.

```
# df_k() and entropy_R2() are defined in survey_lca_source.R; w_vec and the
# sum-to-n scale_ic came from prepare_items() (rationale documented there).
enum <- furrr::future_map_dfr(cfg$K_range, function(K) {
  fit <- fit_lca(dat_prepared, w_vec, cats, cfg$items, K, cfg$n_starts)
  ll_n <- fit$ll * scale_ic # log-likelihood on the sum-to-n scale
  tibble(K = K,
    logLik = ll_n,
    df = df_k(K, cats),
    AIC = -2 * ll_n + 2 * df_k(K, cats),
    BIC = -2 * ll_n + df_k(K, cats) * log(nrow(dat_prepared)),
    entropy = entropy_R2(fit$post, K),
    converged = fit$converged)
```

```

}, .options = furrr::furrr_options(seed = TRUE,
  packages = c("matrixStats", "clue", "purrr", "dplyr", "tidyr")))

# K is the ANALYST's decision (cfg$K_force), made by iterating over the
# enumeration and the per-K diagnostics; nothing is auto-selected.
K_star <- as.integer(cfg$K_force %||% NA)

enum |>
  mutate(across(c(logLik, AIC, BIC), ~ round(.x, 1)),
    entropy = round(entropy, 3)) |>
  knitr::kable(aligned = "r",
    caption = "Weighted fit statistics by number of classes. Selection is by minimum

```

Table 3: Weighted fit statistics by number of classes. Selection is by minimum BIC.

K	logLik	df	AIC	BIC	entropy	converged
2	-31579.7	157	63473.3	64300.0	0.904	TRUE
3	-30176.2	236	60824.3	62067.0	0.913	TRUE
4	-29447.7	315	59525.5	61184.1	0.921	TRUE
5	-29047.7	394	58883.4	60957.9	0.907	TRUE
6	-28751.8	473	58449.5	60940.1	0.895	TRUE
7	-28537.7	552	58179.3	61085.8	0.893	TRUE
8	-28390.2	631	58042.5	61365.0	0.890	TRUE

The selected solution has **5 classes** (forced through `cfg$K_force`, overriding the BIC pick). The figure shows BIC and entropy across the candidate range; BIC is read for selection and entropy for separation.

```

enum |>
  dplyr::select(K, BIC, entropy) |>
  pivot_longer(-K, names_to = "criterion", values_to = "value") |>
  mutate(criterion = recode(criterion, BIC = "Weighted BIC", entropy = "Relative entropy")) |>
  ggplot(aes(K, value)) +
    geom_line(linewidth = 0.6, color = viridisLite::viridis(1, begin = 0.4)) +
    geom_point(size = 2.4, color = viridisLite::viridis(1, begin = 0.4)) +
    facet_wrap(~ criterion, scales = "free_y") +
    scale_x_continuous(breaks = cfg$K_range) +
    labs(x = "Number of latent classes (K)", y = NULL) +
    theme_lca()

```

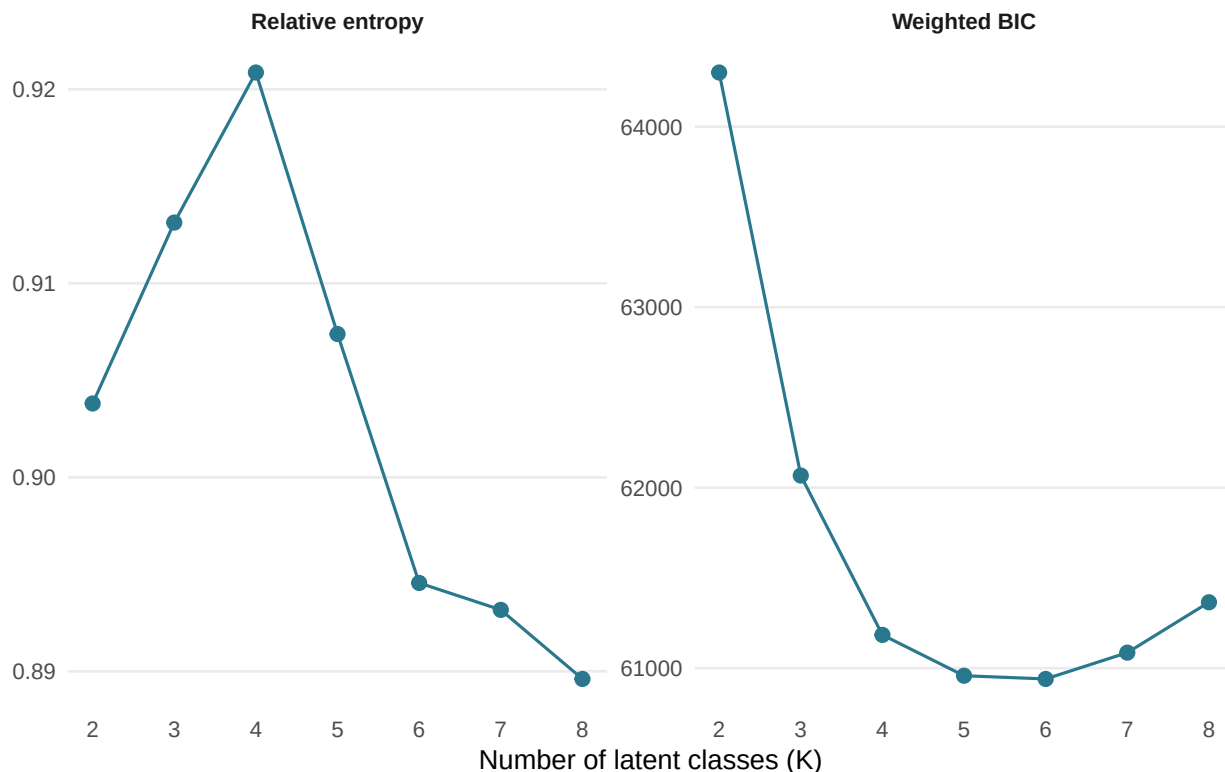


Figure 1: Weighted BIC and relative entropy across candidate numbers of classes. The chosen K minimizes BIC; entropy describes how cleanly respondents are classified.

```
if (!K_ready) cat("**`cfg$K_force` is `NULL`.** Review the enumeration table and figure above,
```

9 The population measurement model

At the selected K , the script fits the model two ways on the same data: **weighted** (the population measurement model) and **unweighted** (which, fit with **poLCA**'s assumptions, is what **poLCA** itself targets). The comparison in Section 12 then shows what the design weights change. *Why this section matters: this is the deliverable. The prevalences and the response-probability profiles are what a reader actually quotes and names, and fixing the label order here is what keeps every subsequent table, figure, and interval referring to the same classes.*

The model has two sets of parameters, and both deserve a plain reading because they are the substantive output of the analysis.

The **inclusion probabilities** π_k are the **class prevalences**: the fraction of the population in each class. They sum to 1 across classes. A prevalence of 0.35 for a class means an estimated 35 percent of the population belongs to that type. These are the headline numbers, the size of each segment.

The **conditional probabilities** ρ_{jck} are **item-response probabilities**: given that a respondent is in class k , the probability they give answer c to item j . For each class and item these sum to 1 across the answer categories. They are how you *characterize and name* a class. If, in one class, the

probability of the most-open answer is high on every negotiation item, that class is the “open” type; if another class concentrates on the guarded answers, that is the “closed” type. Reading down the conditional probabilities for a class is reading its profile.

Class labels are arbitrary. The algorithm could number the classes in any order, so “Class 1” carries no meaning on its own. The script does not impose a substantive order on the classes: with items of differing lengths there is no single scale on which to rank them. What it does do is fix one label order (from the weighted fit) and align the unweighted and **poLCA** fits to it (Section 12), so that “Class 2” refers to the same class in every table and figure. You read each class by its conditional-probability profile; after the diagnostics, the naming section attaches the names (from `outputs/llm_labels.csv` when it exists, otherwise drafted once by the LLM) and applies them to everything downstream.

```
# Fit at K_star on the complete cases, weighted and unweighted. The weighted fit sets
# the label order; the unweighted fit is aligned to it so "Class k" is consistent.
fit_w <- fit_lca(dat_prepared, w_vec, cats, cfg$items, K_star, cfg$n_starts)
fit_u <- align_to(fit_lca(dat_prepared, rep(1, nrow(dat_prepared))), cats, cfg$items,
                 K_star, cfg$n_starts), fit_w)

# Posteriors recomputed from the FINAL parameters, then written back so the same replicate
# design serves both the measurement-model CIs and the profiling regression.
inp_full <- make_inputs(dat_prepared, cfg$items, cats)
post_w <- posterior_of(fit_w$pi, fit_w$rho, inp_full$Y)
colnames(post_w) <- paste0("post_class", seq_len(K_star))
modal_int <- max.col(post_w, ties.method = "first")
dat_prepared <- dat_prepared |>
  bind_cols(as_tibble(post_w)) |>
  mutate(modal_class = factor(modal_int, levels = seq_len(K_star)),
         max_posterior = post_w[cbind(seq_len(n()), modal_int)])
```

10 Design-based confidence intervals via the stratified jackknife

Why this section matters: a prevalence without an interval invites false precision. The stratified jackknife is what turns the point estimates into defensible inference under the actual design, and it is the reason a typology built on few PSUs reports honestly wide intervals rather than misleadingly tight ones.

10.1 Why the survey weights alone are not enough

A natural question at this point is: the data already came with weights, so why not simply use them and be done? The answer is that survey weights do **one** of the two jobs that honest estimation requires, and the confidence intervals depend on the other.

The weights do the first job, **point estimation**: they make the prevalences and response probabilities describe the *population* rather than the achieved sample. That is precisely the weighted

M-step in Section 7, $\hat{\pi}_k = \sum_i w_i p_{ik} / \sum_i w_i$, and the script does exactly that. If the only goal were the estimates themselves, “just use the weights” would be the complete answer.

The weights do **not** do the second job, **variance estimation**, because the precision of a survey estimate depends on *how the sample was selected*, not only on the weights. Two surveys can share an identical weight column yet differ greatly in precision: a sample of 1,200 people drawn as 20 clusters carries far less information than 1,200 people drawn independently, because respondents within a cluster tend to resemble one another, so each additional person in a cluster adds less new information than an independent person would. The weight attached to a respondent says nothing about which cluster they came from; the **design** (the stratum and PSU identifiers) is what carries that information. This is why the code below declares `svydesign(ids = ~psu, strata = ~stratum, weights = ~w)` and builds replicate weights from it, rather than working from the weight column alone.

The practical consequence of ignoring this is concrete and one-directional. If you weighted the data and then computed a standard error as though the weighted sample were a simple random sample of independent people, the estimate would be correct but the standard error would typically be **too small**, often by a factor of 1.5 to 2.5 in clustered samples. That inflation factor is the *design effect*. Using the weights for the estimate while ignoring the design for the variance, the single most common error in survey analysis, produces confidence intervals that are falsely narrow and overstates how much the data actually tell you.

There is a second, independent reason the variance cannot come from the weighted fit alone. The weighted objective $\sum_i w_i \log(\cdot)$ is a *pseudo*-log-likelihood, not a genuine likelihood for any probability model, because fractional and unequally weighted contributions do not correspond to a count of independent observations. The standard maximum-likelihood machinery that would ordinarily hand back standard errors from the curvature of the likelihood (the information matrix) therefore does not apply. The variance has to be estimated by *replication*, refitting the model on systematically perturbed versions of the design, which is what the stratified jackknife does and what the next subsection describes.

In one sentence for a non-technical audience: **the weights tell us who the population is, and the design tells us how sure we are**, and we need both.

10.2 Estimating the design-based variance

Confidence intervals must reflect the sampling design, not pretend the data are a simple random sample. The script uses **replicate weights**: the design is declared, converted to a set of stratified jackknife (JKn) replicates, and the entire weighted EM is re-estimated on each replicate. The spread of the replicate estimates gives the design-based variance,

$$\widehat{\text{Var}}(\hat{\theta}) = \sum_r c_r (\hat{\theta}^{(r)} - \hat{\theta})^2,$$

where each JKn replicate r deletes one primary sampling unit and reweights the rest of its stratum, with the scale factor c_r that **survey** supplies. With about 20 PSUs across 9 strata this produces roughly 20 replicates and about 11 degrees of freedom for the variance, which is the true precision of the design regardless of the 1,200 respondents; intervals will be honestly wide.

Two robustness points are built in. The design declaration uses `nest = TRUE` so PSUs nested within strata are handled correctly. And **lonely PSUs** (a stratum with a single PSU, which would break JK_{nl}) are handled by setting `options(survey.lonely.psu = "adjust")`, which centers such a stratum's contribution at the grand mean rather than erroring; the script reports whether any were encountered. Each replicate refit is **warm-started at the full-sample solution and aligned to it**, which keeps the 20 refits stable, fast, and free of label switching.

```
options(survey.lonely.psu = "adjust")

# Report PSU structure so the analyst sees the degrees of freedom they actually have.
# Note: dplyr::count() is namespace-qualified because matrixStats (loaded for
# rowLogSumExps) also exports a count(), and it loads after dplyr here.
psu_structure <- dat_prepared |>
  dplyr::distinct(.data[[cfg$strata]], .data[[cfg$psu]]) |>
  dplyr::count(.data[[cfg$strata]], name = "psus_in_stratum")
n_psu_total <- nrow(dplyr::distinct(dat_prepared, .data[[cfg$psu]]))
n_strata <- n_distinct(dat_prepared[[cfg$strata]])
lonely_any <- any(psu_structure$psus_in_stratum < 2)

# Declare the design and build stratified jackknife replicate weights.
des <- svydesign(ids = as.formula(paste0("~", cfg$psu)),
  strata = as.formula(paste0("~", cfg$strata)),
  weights = as.formula(paste0("~", cfg$weight)),
  data = dat_prepared, nest = TRUE)
rep_des <- as.svrepdesign(des, type = "JKn")

# Parameter bookkeeping as a TIBBLE, in the exact order flatten_fit() produces.
# This carries item/category/class as real columns, so nothing is recovered from
# string parsing later and the survey-question names are preserved downstream.
param_meta <- bind_rows(
  tibble(kind = "pi", item = NA_character_, category = NA_integer_, class = seq_len(K_star)),
  imap_dfr(cats, function(Cj, item_name)
    expand_grid(class = seq_len(K_star), category = seq_len(Cj)) |> # category varies fastest
    transmute(kind = "rho", item = item_name, category, class))
)
param_names <- param_meta |>
  mutate(nm = if_else(kind == "pi",
    sprintf("pi[%d]", class),
    sprintf("rho[%s,cat%d,class%d]", item, category, class))) |>
  pull(nm)
flatten_fit <- function(fit) set_names(c(fit$pi, unlist(map(fit$rho, as.vector))), param_names)

# The statistic survey::withReplicates evaluates on each replicate: refit the weighted EM
# on the replicate weights, warm-started at the full-sample fit and aligned to it.
theta_fun <- function(w_rep, data) {
  inp <- make_inputs(data, cfg$items, cats)
  fit <- em_run(inp$Y, inp$OH, cats, w_rep, K_star,
    init = fit_w[c("pi", "rho")], maxit = 400L)
```

```

  flatten_fit(align_to(fit, fit_w))
}

rep_var <- withReplicates(rep_des, theta_fun)
est_vec <- flatten_fit(fit_w)
se_vec <- sqrt(diag(vcov(rep_var)))

# Tidy table of every probability with its design-based 95% CI: item/category/class
# are columns, ready to filter, join, and plot.
ci_tbl <- param_meta |>
  mutate(estimate = as.numeric(est_vec), se = as.numeric(se_vec),
         lo = pmax(0, estimate - 1.96 * se),
         hi = pmin(1, estimate + 1.96 * se))

```

The class-size (inclusion) probabilities, with their design-based intervals:

```

ci_tbl |>
  filter(kind == "pi") |>
  transmute(Class = paste("Class", class),
            Prevalence = round(estimate, 3),
            SE = round(se, 4),
            `95% CI` = sprintf("[% .3f, % .3f]", lo, hi)) |>
  knitr::kable(align = "lrrr",
               caption = "Population class prevalences (inclusion probabilities) with design-ba

```

Table 4: Population class prevalences (inclusion probabilities) with design-based JK_n confidence intervals. Classes are numbered here; names are drafted after the diagnostics (Naming the classes).

Class	Prevalence	SE	95% CI
Class 1	0.331	0.0201	[0.292, 0.371]
Class 2	0.111	0.0103	[0.091, 0.132]
Class 3	0.209	0.0191	[0.171, 0.246]
Class 4	0.107	0.0114	[0.084, 0.129]
Class 5	0.242	0.0205	[0.202, 0.282]

The item-response (conditional) probabilities are shown as a heatmap, which is far easier to scan than overlaid lines when there are several items: each panel is a class, each row an item, each column a response category, and the shade (and printed value) is the probability of that answer within that class. Reading a panel top to bottom gives that class's profile; comparing panels shows how the classes differ. Items with fewer categories simply leave the higher columns blank. The design-based confidence interval for every cell is in the exported table `lca_measurement_model_with_CIs.csv`; intervals are omitted from the heatmap only to keep it legible.

```

rho_plot <- ci_tbl |>
  filter(kind == "rho") |>
  mutate(Class = factor(paste("Class", class), levels = paste("Class", seq_len(K_star))),
         item = factor(item, levels = cfg$items),
         category = factor(category),
         lbl = sprintf("%.2f", estimate),
         lbl_col = if_else(estimate > 0.55, "white", "grey15"))

ggplot(rho_plot, aes(category, fct_rev(item), fill = estimate)) +
  geom_tile(color = "white", linewidth = 0.5) +
  geom_text(aes(label = lbl, color = lbl_col), size = 2.4) +
  scale_color_identity() +
  facet_wrap(~ Class, nrow = 1) +
  scale_fill_viridis_c(limits = c(0, 1), name = "P(response | class)") +
  labs(x = "Response category", y = NULL) +
  theme_lca() +
  theme(panel.grid = element_blank(),
        legend.key.width = unit(1.4, "cm"))

```

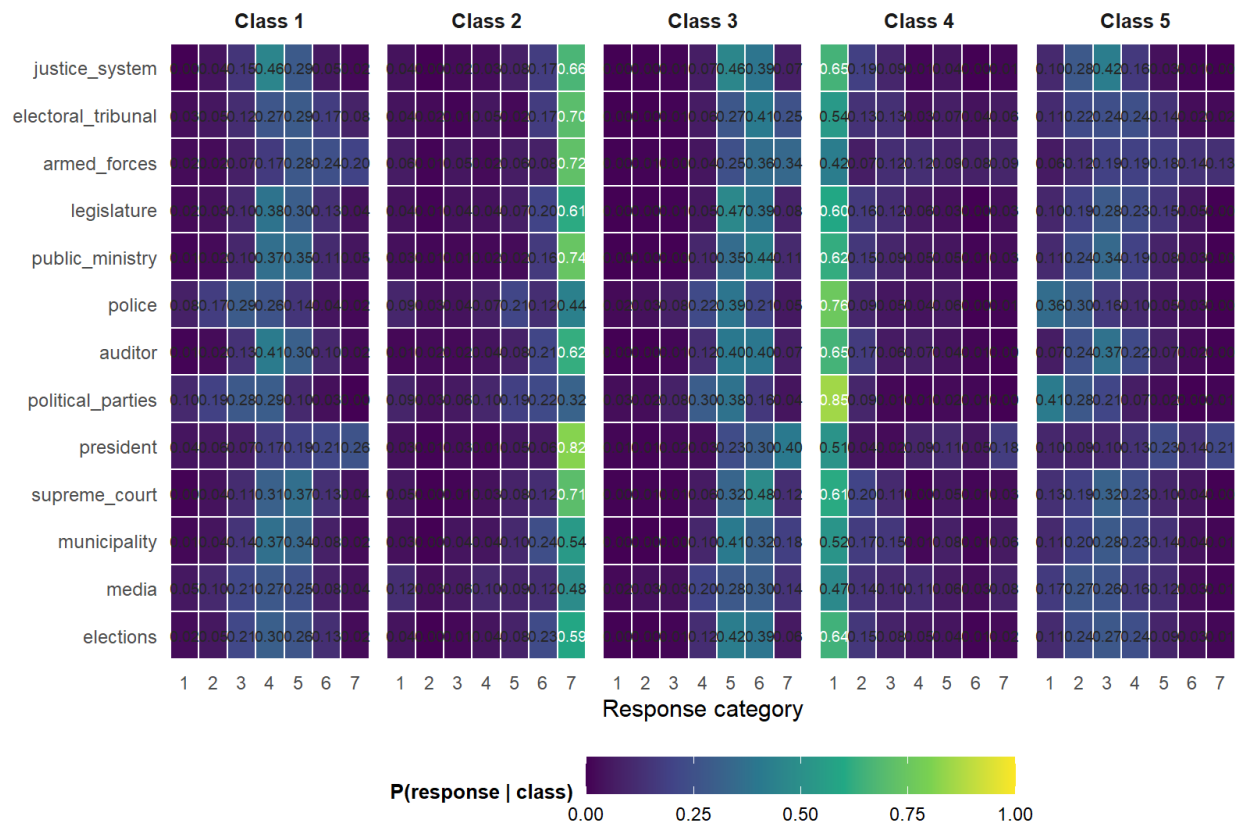



Figure 2: Population item-response (conditional) probabilities by class. Each panel is a class; rows are items, columns are response categories. Items of different lengths leave the higher columns blank. Cell shade and label give $P(\text{response} \mid \text{class})$. Panels are numbered; names are drafted after the diagnostics.

11 Which indicators carry the classification

Not every item earns its place in the model. An item helps define the classes only to the extent that the classes answer it *differently*; an item that every class answers the same way adds parameters and noise without sharpening the typology, and is a candidate to drop. The diagnostic below reads this directly off the fitted measurement model (the same ρ shown in the heatmap above).

For each item it computes a **discrimination** score: the average total-variation distance between the class-conditional response distributions, taken over all pairs of classes. For two classes a and b the total-variation distance on item j is $\frac{1}{2} \sum_c |\rho_{jca} - \rho_{jcb}|$, which is 0 when the two classes answer identically and 1 when their answers never overlap. Averaging over class pairs gives one number per item in $[0, 1]$: near 0 means the item does not separate the classes, larger means it does. The low items are the ones to consider removing; after dropping them you refit and confirm the class structure and its interpretation are essentially unchanged.

The discrimination score is an effect size; on its own it does not say how small is “too small.” The automated screen below turns it into a keep-or-drop decision with BIC. For each item it asks whether letting that item’s response probabilities differ across classes is worth the extra parameters, by

comparing the fitted model against a restricted model in which the item is made *class-independent* (its response distribution shared across classes, so it cannot inform the classes). When BIC prefers the restricted model, the item is flagged as a weak contributor. This is the model-comparison logic of Dean and Raftery (2010, *Annals of the Institute of Statistical Mathematics* 62(1):11–35) in a simplified form. Their method embeds the same full-versus-restricted BIC comparison inside a stepwise (headlong) search over which variables to keep; the screen here runs it once per item against the full model rather than as a search. The `LCAvarsel` package implements the further refinement of Fop, Smart, and Murphy (2017, *Annals of Applied Statistics* 11(4):2080–2110), which lets an excluded item depend on the other items instead of collapsing to its marginal as the simple restricted model here does. The size of the BIC gap grades the strength of the evidence: by Kass and Raftery (1995, *Journal of the American Statistical Association* 90(430):773–795), a difference beyond about 6 is strong and beyond about 10 very strong.

```
# Discrimination of each item under the fitted weighted model: how differently the classes
# answer it, measured as the mean over class pairs of the total-variation (TV) distance
# between their class-conditional response distributions. 0 = all classes answer alike (a
# drop candidate); near 1 = strong separation. Loop-free: map over items, then over pairs.
# Computed here; shown in the figure below and in the screen table further down.
class_pairs <- utils::combn(K_star, 2, simplify = FALSE) # list of c(a, b) class-index pairs

item_discrim <- tibble(
  item = cfg$items,
  discrimination = map_dbl(fit_w$rho, function(rho_j) {
    # rho_j is (categories x classes); rho_j[, k] is class k's response distribution.
    # TV distance for one pair of classes, then averaged over all pairs.
    mean(map_dbl(class_pairs, ~ 0.5 * sum(abs(rho_j[, .x[1]] - rho_j[, .x[2]]))))
  })
) |>
  arrange(desc(discrimination))

item_discrim |>
  mutate(item = factor(item, levels = rev(item))) |>
  ggplot(aes(discrimination, item)) +
  geom_col(fill = viridisLite::viridis(1, begin = 0.45), width = 0.7) +
  geom_vline(xintercept = 0.1, linetype = "dashed", color = "grey55") +
  scale_x_continuous(limits = c(0, 1)) +
  labs(x = "Discrimination (mean pairwise total-variation distance)", y = NULL) +
  theme_lca()
```

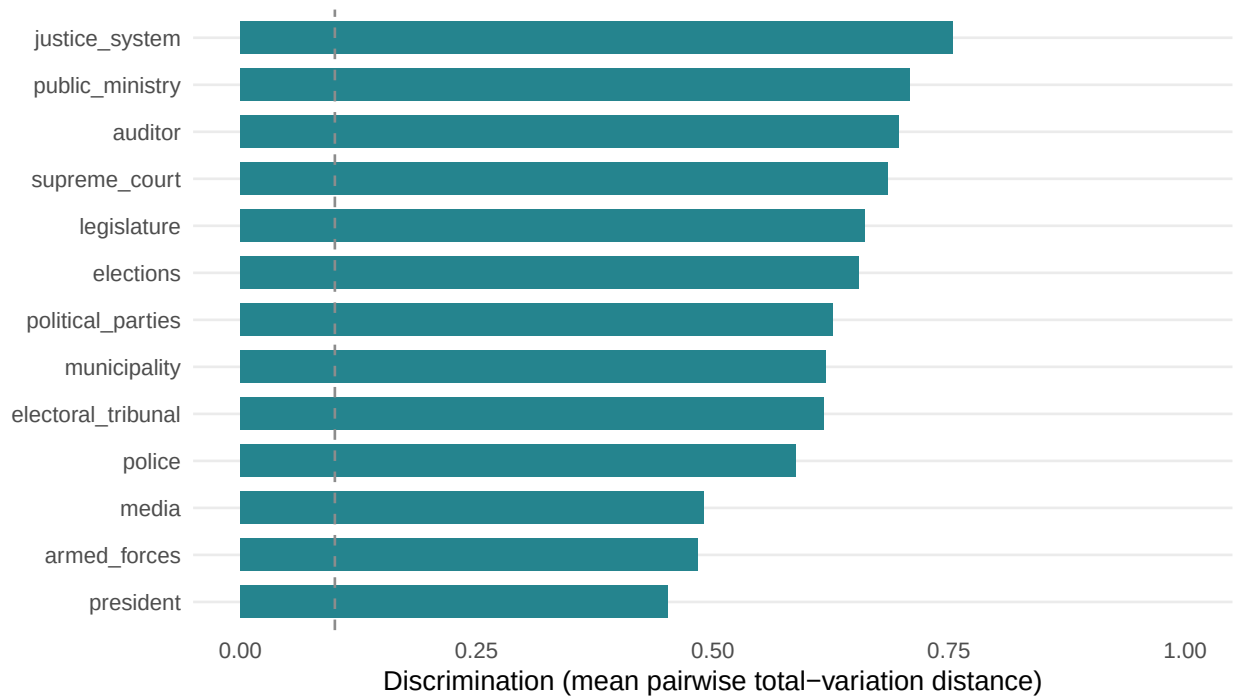


Figure 3: Item discrimination: how differently the latent classes answer each item (mean pairwise total-variation distance of the class-conditional response distributions). Bars near zero mark items that barely separate the classes and are candidates to drop before refitting. The dashed line at 0.1 is a soft reference, not a hard cutoff.

```
# Automated screen: does each item earn its class-DEPENDENT parameters by BIC? Compare, on
# the SAME respondents, the fitted model against a restricted model in which item j is made
# class-INDEPENDENT (its response distribution shared across classes, so it tells us nothing
# about the latent classes). Under local independence that restricted model factorises into
# an LCA on the OTHER items times item j's marginal, so its weighted pseudo-log-likelihood
# is ll(LCA without item j) + ll(item j's weighted marginal), and its BIC is directly
# comparable to the full model's. delta_bic = BIC(restricted) - BIC(full): NEGATIVE means
# BIC prefers dropping item j's class-dependence, so the item is flagged.
#
# This is the slowest chunk: it refits the LCA once per item. Lower cfg$n_starts here if you
# need speed, at some risk of a local optimum. It is reproducible because fit_lca() seeds
# each start from cfg$seed.
n_eff <- nrow(dat_prepared)
# Put every log-likelihood on the same sum-to-n scale used for the criteria in
# Section 8 (scale_ic = n / sum(w_vec)), so the screen's BIC comparison is
# calibrated the same way as the K-selection BIC.
ll_full <- fit_w$ll * scale_ic
df_full <- df_k(K_star, cats) # df_k() was defined in Section 8

# Weighted log-likelihood of one item under a single (class-independent) distribution.
marginal_ll <- function(item) {
  y <- as.integer(dat_prepared[[item]])
```

```

  p <- as.numeric(xtabs(w_vec ~ y)); p <- p / sum(p) # weighted response proportions, 1..C
  sum(w_vec * log(p[y]))
}

bic_screen <- tibble(item = cfg$items) |>
  mutate(delta_bic = furrr::future_map_dbl(cfg$items, function(j) {
    keep      <- setdiff(cfg$items, j)
    fit_wo    <- fit_lca(dat_prepared, w_vec, cats[keep], keep, K_star, cfg$n_starts)
    ll_restr  <- (fit_wo$ll + marginal_ll(j)) * scale_ic # sum-to-n scale, as above
    df_restr  <- df_k(K_star, cats[keep]) + (cats[[j]] - 1) # item j: one shared distribution
    (-2 * ll_restr + df_restr * log(n_eff)) - (-2 * ll_full + df_full * log(n_eff))
  }), .options = furrr::furrr_options(seed = TRUE,
    packages = c("matrixStats", "clue", "purrr", "dplyr", "tidyr"))))

# Combine the effect size and the decision into one ranked, flagged table.
item_screen <- item_discrim |>
  left_join(bic_screen, by = "item") |>
  mutate(flag = if_else(delta_bic < 0, "drop candidate", "keep")) |>
  arrange(delta_bic)

item_screen |>
  mutate(discrimination = round(discrimination, 3), delta_bic = round(delta_bic, 1)) |>
  knitr::kable(align = "lrrl",
    col.names = c("Item", "Discrimination", "Delta BIC", "Screen"),
    caption = "Automated indicator screen, most-droppable first. Discrimination is t")

```

Table 5: Automated indicator screen, most-droppable first. Discrimination is the mean pairwise total-variation distance (effect size). Delta BIC is BIC(item made class-independent) minus BIC(full model); a negative value means BIC prefers dropping the item's class-dependence, so it is flagged. The more negative the gap, the stronger the case (beyond about 6 is strong, beyond 10 very strong).

Item	Discrimination	Delta BIC	Screen
president	0.452	320.0	keep
armed_forces	0.484	424.0	keep
media	0.490	490.7	keep
police	0.588	742.1	keep
electoral_tribunal	0.618	827.5	keep
political_parties	0.627	863.6	keep
municipality	0.620	890.7	keep
legislature	0.661	987.7	keep
elections	0.654	988.1	keep
supreme_court	0.685	1116.4	keep
public_ministry	0.708	1161.9	keep
auditor	0.697	1197.1	keep
justice_system	0.754	1419.3	keep

Treat a flag as a prompt, not a verdict. An item can be statistically weak yet worth keeping for face validity, and a binary item tends to score lower simply because it has less room to separate classes. Drop the items that are both flagged and substantively dispensable, set `cfg$items` to the survivors, refit, and confirm the prevalences and class profiles are essentially unchanged; if they move a lot, the item was carrying real information and belongs in the model.

11.1 Local independence: the bivariate residual

The screen above asks whether each item *contributes* to the classes. A different failure is two items being *locally dependent*: more associated with each other than shared class membership explains, which violates the conditional-independence assumption the whole model is built on. The **bivariate residual (BVR)** diagnoses this per item pair. For items a and b it compares the weighted observed two-way table with the table the fitted model implies,

$$p_{rs}^{\text{exp}} = \sum_{k=1}^K \pi_k \rho_{a,rk} \rho_{b,sk},$$

as a Pearson X^2 scaled by n and divided by the table's $(C_a - 1)(C_b - 1)$ degrees of freedom, so pairs with different category counts are comparable.

Two honesty notes before the table. First, under a weighted pseudo-likelihood on a complex design the usual chi-square reference for X^2 does not hold, so the BVR here is a **descriptive ranking index, not a test**; read the *relative* sizes. Absolute cutoffs from the simple-random-sampling literature (values above roughly 3) do not transfer here, because unequal weights inflate the index above its nominal scale by roughly the weight-variation design effect; the operative signal is a pair standing well above the rest of its own table. Second, the residual is computed against the fitted K -class model, so it inherits that choice: a pair can look dependent simply because K is too small.

```
# bvr_pairs() is defined in survey_lca_source.R; fit$rho is positional in cfg$items
# order, so the items are passed in exactly that order.
bvr_tab <- bvr_pairs(dat_prepared, w_vec, cfg$items, fit_w)

bvr_tab |>
  mutate(bvr = round(bvr, 2)) |>
  slice_head(n = 10) |>
  knitr::kable(align = "llrr",
               col.names = c("Item A", "Item B", "Pair df", "BVR"),
               caption = "Largest bivariate residuals (BVR), a per-pair local-independence index")
```

Table 6: Largest bivariate residuals (BVR), a per-pair local-independence index: the design-weighted observed two-way table against the model-implied table, as Pearson $X^2 / (n - \text{scaled})$ per degree of freedom. Descriptive ranking only; no chi-square test applies under the weighted pseudo-likelihood. Pairs standing well above the rest of the table mark associations the classes do not explain; absolute SRS cutoffs do not transfer under unequal weights.

Item A	Item B	Pair df	BVR
public_ministry	auditor	36	5.75
armed_forces	legislature	36	5.53
justice_system	supreme_court	36	3.21
police	political_parties	36	2.81
president	supreme_court	36	2.48
president	elections	36	2.40
electoral_tribunal	public_ministry	36	2.36
police	municipality	36	2.32
justice_system	elections	36	2.19
armed_forces	police	36	2.14

How to act on it, in order of preference. A pair that stands far above the rest and is *substantively* redundant (two near-identical wordings of one question) is a case for dropping one of the pair; rerun the indicator screen after, since removing one member changes the other’s contribution. If the pair is substantively distinct, the dependence may instead be a sign that K is too small, so check whether the pair’s BVR falls at $K + 1$ before removing anything; a residual that melts away with one more class was class structure, not item redundancy. Only if it persists and both items must stay for content reasons does it become a documented limitation, a known local-independence violation whose main cost is overstated classification certainty for respondents whose pattern rides on that pair. A flat BVR table with no outlying pairs is affirmative evidence for the conditional-independence assumption and worth stating in the write-up.

12 Naming the classes

Class numbers carry no meaning, so this section attaches a short label and a one-to-two-sentence description to each class and applies them to **every table and figure from here on** (the diagnostics above deliberately used class numbers). One rule: when `outputs/llm_labels.csv` exists it is used (validated against `cfg$K_force`); when it does not, the LLM drafts it once, one call per class, from that class’s response probabilities and the item wording (each item’s response labels come from its own observed values, a value’s label when one exists and its number otherwise, so differing scales, anchors-only labeling, and unlabeled items all work; wording comes from the dictionary), and writes the file. Editing that file IS taking over the naming. `cfg$survey_context`, one to two paragraphs on the survey’s collection and purpose, is prepended so the model can resolve referents like “the agreement” in the item wording; when set, rule 1 gains a fence restricting the context to referent resolution, so nothing may be attributed to a class that its probabilities do not show, and when NULL the prompt is byte-identical to the certified context-free version.

The one-call-per-class design is empirical, not stylistic: in validation experiments, a single prompt carrying all classes systematically confused the closest class pairs, because a joint one-to-one assignment lets one confusion corrupt two classes, and a two-stage variant that fed a global comparison into each per-class call changed nothing. Isolated calls cost no accuracy and give one clean, independently frozen result per class. The provider is one OpenAI-compatible branch: `cfg$compass_base_url` with `cfg$llm_model`, your endpoint at work, OpenRouter at home, and no API key appears anywhere in this project (`.Rprofile/.Renv` supply it).

Two safeguards close the design. First, **harmonization**: isolation’s one blind spot is that two near-neighbor classes can draft the same label, since neither call saw the other. When a mechanical collision check fires (duplicate or heavily word-overlapping labels), one closing call sees only the finished label-description pairs, never the profiles, and edits **labels only**, minimally, anchored to each class’s own description; descriptions are frozen, non-colliding labels stay verbatim, and the pre-edit drafts are kept in a `Label_draft` column of the freeze file so every harmonizer change is auditable. When no collision exists, no extra call is made. Second, **determinism**: every class index is pinned by `cfg$seed` (each EM start re-seeds from it, so the classes come out in the same order on every run of the same data), and the same seed with temperature 0 is passed to the LLM. Cloud sampling is not bit-reproducible even so, which is exactly why the labels are frozen to CSV after the first run and reloaded thereafter: the freeze, not the seed, is what guarantees the class-label binding never drifts across renders.

Why this section matters: the labels are the words every reader will quote, and they are drafts, not findings. The LLM reads reliably but names loosely for near-neighbor or diffuse profiles, so verify each label against its response profile in the heatmap above, and prefer editing the frozen CSV over trusting the first draft.

```
# get_class_labels() implements the one rule above (survey_lca_source.R,
# Section 4). The cache file freezes the LLM output for reproducibility.
# cfg$data, NOT dat_prepared: recode_consecutive() strips attributes, so a
# recoded copy would silently cost the prompts all wording and labels.
class_labels <- get_class_labels(fit_w, survey_dat, cfg$items, cfg,
                                questions = set_names(dictionary$question,
                                                        dictionary$item))

knitr::kable(class_labels,
              caption = "Class labels and descriptions in use (outputs/llm_labels.csv). Verify each
```

Table 7: Class labels and descriptions in use (outputs/llm_labels.csv). Verify each against its response profile before quoting; edit the file to take over naming.

K	Label	Description
1	Moderate Institutional Trust	This class shows a tendency toward moderate trust (rating 4) across most institutions, with notably higher trust in the president and armed forces, and lower trust in the police and political parties.
2	Maximum Institutional Trust	This class is characterized by very high probabilities of reporting maximum trust (7) across nearly all institutions, most notably the president (0.82), public ministry (0.74), and armed forces (0.72).

K	Label	Description
3	High Institutional Trust	This class is characterized by high levels of trust across all institutions, with the strongest probabilities concentrated on ratings of 5, 6, or 7, particularly for the president, armed forces, and supreme court.
4	Generalized Institutional Distrust	This class is characterized by very high probabilities of reporting the lowest level of trust (1) across nearly all institutions, most notably for political parties (85%) and the police (76%).
5	Presidential and Military Trust	This class is characterized by high distrust of political parties and police, while showing relatively higher trust in the president and armed forces compared to other institutions.

```
# Display labels for every downstream table and figure: short, wrapped, fixed
# class order so factor levels are stable everywhere.
# Three display tiers, because one label format cannot serve every surface:
#   lab_of      wrapped factor, for FIGURE legends and facets (newlines are fine there)
#   lab_line    single line "Ck: Label", for SMALL tables (a newline inside a kable
#               cell breaks Typst layout)
#   lab_short   "Ck" only, for LARGE tables; the class-label table above is the legend
lab_lev  <- stringr::str_wrap(paste0("C", class_labels$K, ": ", class_labels$Label), 22)
lab_of   <- function(k) factor(lab_lev[k], levels = lab_lev)
lab_line <- function(k) paste0("C", k, ": ", class_labels$Label[k])
lab_short <- function(k) paste0("C", k)
```

13 Benchmark against poLCA, and a validation check

Two comparisons with poLCA close the loop. First, a **validation**: when all weights are set to one, the weighted EM and poLCA maximize the identical likelihood, so their estimates must agree up to label order and convergence tolerance. Confirming this is the strongest evidence that the custom EM is implemented correctly. Second, a **substantive benchmark**: poLCA fit normally (unweighted) estimates the achieved-sample structure, and its gap from the weighted estimates measures how much the design matters for this typology. The script also reports the K that poLCA's own BIC would select, since informative sampling can change the apparent number of classes. *Why this section matters: it does double duty. The unit-weight check is the script's correctness test against an independent, trusted implementation, and the weighted-versus-poLCA gap is the headline quantity on real data, the visible imprint of the design on the class shares, which is what you keep when no ground truth exists.*

```
# poLCA needs a data frame of items coded 1..C with no NA (it uses na.rm).
polca_df <- dat_prepared |> dplyr::select(all_of(cfg$items)) |> as.data.frame()
polca_f  <- as.formula(paste0("cbind(", paste(cfg$items, collapse = ", "), ") ~ 1"))

# --- substantive benchmark: poLCA at K_star, unweighted ---
set.seed(cfg$seed + 7L)
pl <- poLCA(polca_f, data = polca_df, nclass = K_star, nrep = 10,
            maxiter = 5000, na.rm = TRUE, verbose = FALSE)
```



```

pl_aligned <- align_to(list(pi = as.numeric(pl$P),
                           rho = map(pl$probs, t),
                           post = pl$posterior), fit_w)

# --- poLCA's own BIC-selected K across the candidate range ---
polca_bic <- furrr::future_map_dfr(cfg$K_range, function(K) {
  set.seed(cfg$seed + 10L + K)
  m <- poLCA(polca_f, data = polca_df, nclass = K, nrep = 5,
             maxiter = 5000, na.rm = TRUE, verbose = FALSE)
  tibble(K = K, poLCA_BIC = m$bic)
}), .options = furrr::furrr_options(seed = TRUE, packages = "poLCA"))
polca_K <- polca_bic |> slice_min(poLCA_BIC, n = 1) |> pull(K) |> first()

# --- validation: weighted EM with all weights = 1 should match poLCA ---
fit_unit <- align_to(fit_lca(dat_prepared, rep(1, nrow(dat_prepared)), cats, cfg$items,
                           K_star, cfg$n_starts), fit_w)
max_pi_gap <- max(abs(fit_unit$pi - pl_aligned$pi))
max_rho_gap <- max(map2_dbl(fit_unit$rho, pl_aligned$rho, ~ max(abs(.x - .y))))

tibble(`Latent class` = lab_line(seq_len(K_star)),
       `Population (weighted)` = round(fit_w$pi, 3),
       `Sample (poLCA)` = round(pl_aligned$pi, 3),
       `Difference (pp)` = round((fit_w$pi - pl_aligned$pi) * 100, 1)) |>
  knitr::kable(align = "lrrr",
               caption = "Class prevalence: weighted (population) versus unweighted poLCA (achieved sample)")

```

Table 8: Class prevalence: weighted (population) versus unweighted poLCA (achieved sample). The difference is the imprint of the design.

Latent class	Population (weighted)	Sample (poLCA)	Difference (pp)
C1: Moderate Institutional Trust	0.331	0.336	-0.5
C2: Maximum Institutional Trust	0.111	0.111	0.0
C3: High Institutional Trust	0.209	0.207	0.1
C4: Generalized Institutional Distrust	0.107	0.107	0.0
C5: Presidential and Military Trust	0.242	0.239	0.4

The validation check returns a maximum class-size discrepancy of 0.0049 and a maximum response-probability discrepancy of 0.0085 between the unit-weight EM and poLCA; values near zero confirm the EM is correct. The weighted BIC selected 5 classes, and poLCA's unweighted BIC selected 6.

```

bind_rows(
  tibble(Class = lab_of(seq_len(K_star)), prevalence = fit_w$pi,
         Estimator = "Weighted (population)"),
  tibble(Class = lab_of(seq_len(K_star)), prevalence = pl_aligned$pi,

```

```

    Estimator = "Unweighted (poLCA)")
) |>
  ggplot(aes(Class, prevalence, fill = Estimator)) +
  geom_col(position = position_dodge(width = 0.7), width = 0.6) +
  scale_fill_manual(values = wu_pal) +
  scale_y_continuous(limits = c(0, NA)) +
  labs(x = NULL, y = "Class prevalence", fill = NULL) +
  theme_lca()

```



Figure 4: Population (weighted) versus achieved-sample (poLCA) class prevalences. Separation between the two indicates an informative design for this typology.

14 Membership assignment and the SPSS handoff

Membership is attached to the preserved original frame and written as a labelled SPSS file for analysts, only when `cfg$K_force` is set and the class-label file exists, so the assignments always carry the final labels. The analysis used complete cases; a posterior class requires all items, so cases with any missing item receive NA (there is no honest assignment without their answers), and the `keep` flag in the file records exactly who was assigned.

```

label_file <- file.path(cfg$out_dir, "llm_labels.csv")
if (file.exists(label_file)) {

```

```

assigned <- max.col(fit_w$post, ties.method = "first") # rows = survey_dat rows
out_sav <- survey_dat_original |>
  mutate(class = NA_integer_,
         class = replace(class, keep, assigned),
         class = haven::labelled(
           class, labels = set_names(class_labels$K, class_labels$Label),
           label = "Latent class (design-weighted LCA)")
  )
haven::write_sav(out_sav, file.path(cfg$out_dir, "survey_with_classes.sav"))
cat("Wrote ", file.path(cfg$out_dir, "survey_with_classes.sav"), ": ",
    sum(!is.na(out_sav$class)), " assigned, ",
    sum(is.na(out_sav$class)), " NA (incomplete items).\n", sep = "")
} else {
  cat("Class-label file not found at", label_file,
      "\n- run the labeling section first; the export uses those labels.\n")
}

```

Wrote D:/repos/weighted_and_biased/LCA/outputs/survey_with_classes.sav: 1430 assigned, 192 NA

15 Profiling class membership on demographics

The posterior class probabilities were written back onto the data in Section 9, so each respondent now carries a full posterior over the classes. Per the analysis plan, no posterior threshold is applied: every respondent is kept, so the profiling sample is the full design sample and the weights retain their meaning. The task now is to describe *who* falls in each class, by demographics, in a way that is design-correct and reads directly as a campaign-usable quantity.

15.1 Which demographics belong here

Any demographic may appear here except the survey items that defined the classes. A variable used to build the survey weights, such as age, is fine to profile on and is not double-counting: the weights make each subpopulation estimate represent the population, and the demographic simply describes that represented population. The circularity to avoid is a different one: a variable used as an **indicator** of the latent classes must not also be profiled as if it described them, because that would explain the classes with the very items that built them. Demographics used in weighting are fine; the survey items that built the typology are not.

15.2 Design-based class prevalence by demographic

Rather than model class membership with a multinomial regression on the demographics, the profile reports, for each demographic level, the **design-based prevalence of each class within that subpopulation**: the share of that group estimated to belong to each type. This is a domain (subpopulation) estimate, the most standard quantity in survey sampling, and it is the right tool even when a demographic coincides with the design: a prevalence **within** a stratifying variable's levels

is a well-defined quantity, whereas a regression coefficient on that variable would be confounded with the stratification and unstable.

The class-membership target is the **posterior probability** of each class, not a hard modal assignment. The design-weighted mean of the posterior within a subpopulation is that subpopulation's class prevalence, and because the design-weighted posterior mean over the whole sample reproduces the overall prevalences π_k from Section 9, the demographic breakdown is coherent with the headline numbers rather than attenuated by hard classification. Uncertainty is design-based from the same JKn replicate weights used everywhere else: the prevalences are recomputed on each replicate and `survey::withReplicates` turns the spread into standard errors.

Why this section matters: this is the campaign-facing output. A class prevalence by demographic level, with a design-correct interval, is exactly what targeting needs (“this group is more this type than that group”), and computing it as a domain estimate keeps it stable and interpretable where a regression on a stratifying demographic would not be. The interval respects the design’s degrees of freedom, so a breakdown built on few PSUs per group reports honestly wide bands rather than false precision.

```
# The class-membership target is the soft posterior, already on dat_prepared and
# carried by the Section 10 replicate design (rep_des). post_cols are the K
# posterior columns in class order. The design objects des (the sampling design)
# and rep_des (its JKn replicates) are reused from Section 10 unchanged.
post_cols <- paste0("post_class", seq_len(K_star))

# Fixed layout for the flattened statistic: every (demographic, level, class) cell,
# levels in factor order, class varying fastest within a level. dom_meta names and
# orders the estimate vector so nothing is recovered by string parsing later.
dom_meta <- map_dfr(cfg$aux, function(v)
  expand_grid(plevel = levels(dat_prepared[[v]]), class = seq_len(K_star)) |>
  mutate(pvar = v) |>
  dplyr::select(pvar, plevel, class) |>
  mutate(idx = row_number())

# The statistic survey::withReplicates evaluates on each replicate. For each
# demographic it forms a level-indicator matrix, then the weighted mean of each
# class posterior within each level: crossprod(M, w * P) / crossprod(M, w). The
# result is flattened to ONE named numeric vector in dom_meta's order (level-major,
# class-minor), because withReplicates needs a vector to build the covariance.
domain_theta <- function(w_rep, data) {
  P <- as.matrix(data[, post_cols, drop = FALSE]) # n x K, fixed across replicates
  wP <- w_rep * P # n x K, weighted posteriors
  flat <- map(cfg$aux, function(v) {
    f <- factor(data[[v]], levels = levels(dat_prepared[[v]]))
    M <- model.matrix(~ f - 1) # n x L, one column per level
    num <- crossprod(M, wP) # L x K: weighted class sums per level
    den <- as.numeric(crossprod(M, w_rep)) # L: weighted total per level
    as.vector(t(num / den)) # level-major, class-minor
  })
  unlist(flat, use.names = FALSE)
```

```

}

dom_rep <- withReplicates(rep_des, domain_theta)      # design-based covariance of every cell
dom_est <- domain_theta(weights(des), dat_prepared)  # full-sample point estimates
dom_V    <- vcov(dom_rep)                          # full covariance, for the contrasts b
dom_se    <- sqrt(diag(dom_V))
stopifnot(length(dom_est) == nrow(dom_meta))

# Per-level class prevalence with a design-based interval. The interval is formed
# on the logit scale and back-transformed so it stays inside [0, 1]; the standard
# error is mapped through the delta method, se_logit = se / (p (1 - p)). Estimates
# are clamped a hair off 0 and 1 only to keep the transform finite.
dom_tbl <- dom_meta |>
  mutate(p = as.numeric(dom_est), se = as.numeric(dom_se)) |>
  mutate(p_c = pmin(pmax(p, 1e-6), 1 - 1e-6),
         gse = se / (p_c * (1 - p_c)),
         lo  = plogis(qlogis(p_c) - 1.96 * gse),
         hi  = plogis(qlogis(p_c) + 1.96 * gse)) |>
  dplyr::select(pvar, plevel, class, p, se, lo, hi)

# One compact table PER demographic: fits the page regardless of level-name
# length or K. The class-label table above is the legend for C1..CK.
walk(cfg$aux, function(v) {
  dom_tbl |>
    dplyr::filter(pvar == v) |>
    transmute(Level = plevel, Class = lab_short(class),
              cell = sprintf("%.2f [%.2f, %.2f]", p, lo, hi)) |>
    pivot_wider(names_from = Class, values_from = cell) |>
    knitr::kable(align = str_c("l", strrep("r", K_star)),
                 caption = paste0("Class prevalence within ", v,
                                   " (design-weighted mean posterior; 95% JKn intervals on the l
    print()
    cat("\n\n")
})

```

Table 9: Class prevalence within Employment (design-weighted mean posterior; 95% JKn intervals on the logit scale). Each row sums to one across the class columns.

Level	C1	C2	C3	C4	C5
18-29	0.36 [0.32, 0.40]	0.09 [0.07, 0.12]	0.21 [0.17, 0.24]	0.09 [0.07, 0.12]	0.25 [0.22, 0.28]
30-44	0.32 [0.28, 0.37]	0.10 [0.07, 0.13]	0.21 [0.17, 0.26]	0.11 [0.08, 0.14]	0.26 [0.21, 0.31]
45-59	0.32 [0.28, 0.37]	0.12 [0.09, 0.16]	0.17 [0.14, 0.22]	0.12 [0.09, 0.16]	0.26 [0.22, 0.30]
60+	0.30 [0.25, 0.36]	0.15 [0.11, 0.20]	0.26 [0.21, 0.31]	0.11 [0.08, 0.15]	0.18 [0.13, 0.24]

Table 10: Class prevalence within Employment (design-weighted mean posterior; 95% JK_n intervals on the logit scale). Each row sums to one across the class columns.

Level	C1	C2	C3	C4	C5
Female	0.34 [0.31, 0.38]	0.11 [0.09, 0.14]	0.19 [0.16, 0.22]	0.12 [0.10, 0.14]	0.24 [0.21, 0.27]
Male	0.32 [0.29, 0.35]	0.11 [0.09, 0.13]	0.23 [0.20, 0.26]	0.10 [0.08, 0.12]	0.24 [0.21, 0.28]

Table 11: Class prevalence within Employment (design-weighted mean posterior; 95% JK_n intervals on the logit scale). Each row sums to one across the class columns.

Level	C1	C2	C3	C4	C5
higher_ed	0.32 [0.27, 0.37]	0.07 [0.04, 0.11]	0.17 [0.13, 0.21]	0.10 [0.07, 0.14]	0.34 [0.29, 0.40]
none	0.11 [0.03, 0.29]	0.41 [0.23, 0.62]	0.15 [0.05, 0.33]	0.21 [0.09, 0.42]	0.13 [0.04, 0.31]
primary	0.33 [0.28, 0.40]	0.17 [0.13, 0.22]	0.22 [0.16, 0.28]	0.09 [0.06, 0.14]	0.19 [0.14, 0.24]
secondary	0.34 [0.31, 0.37]	0.10 [0.08, 0.12]	0.22 [0.20, 0.25]	0.11 [0.09, 0.13]	0.23 [0.20, 0.26]

Table 12: Class prevalence within Employment (design-weighted mean posterior; 95% JK_n intervals on the logit scale). Each row sums to one across the class columns.

Level	C1	C2	C3	C4	C5
rural	0.35 [0.30, 0.40]	0.12 [0.09, 0.16]	0.25 [0.20, 0.31]	0.09 [0.06, 0.14]	0.19 [0.15, 0.23]
urban	0.33 [0.30, 0.35]	0.11 [0.09, 0.13]	0.20 [0.17, 0.23]	0.11 [0.09, 0.13]	0.25 [0.23, 0.28]

Table 13: Class prevalence within Employment (design-weighted mean posterior; 95% JK_n intervals on the logit scale). Each row sums to one across the class columns.

Level	C1	C2	C3	C4	C5
Employed	0.33 [0.30, 0.36]	0.10 [0.08, 0.12]	0.20 [0.17, 0.23]	0.12 [0.10, 0.14]	0.24 [0.21, 0.27]
house_keeper	0.32 [0.27, 0.37]	0.13 [0.10, 0.17]	0.22 [0.18, 0.27]	0.09 [0.07, 0.12]	0.24 [0.20, 0.28]
Retired	0.34 [0.26, 0.43]	0.17 [0.10, 0.26]	0.19 [0.13, 0.27]	0.10 [0.05, 0.17]	0.20 [0.14, 0.29]
Student	0.38 [0.30, 0.47]	0.09 [0.05, 0.16]	0.25 [0.18, 0.34]	0.07 [0.03, 0.16]	0.21 [0.14, 0.29]
Unemployed	0.32 [0.24, 0.41]	0.09 [0.05, 0.15]	0.19 [0.12, 0.28]	0.09 [0.05, 0.16]	0.31 [0.23, 0.40]

```

# Horizontal orientation: long level names read cleanly on the y axis, and the
# dodge separates classes without angled-text tricks.
dom_tbl |>
  mutate(Class = lab_of(class)) |>
  ggplot(aes(p, plevel, color = Class)) +
  geom_pointrange(aes(xmin = lo, xmax = hi), size = 0.35, linewidth = 0.4,
    position = position_dodge(width = 0.6)) +
  facet_wrap(~ pvar, scales = "free_y", ncol = 2) +
  scale_color_viridis_d(begin = 0.1, end = 0.8) +
  scale_x_continuous(limits = c(0, NA)) +
  labs(x = "Prevalence of class membership", y = NULL, color = NULL) +
  theme_lca()

```

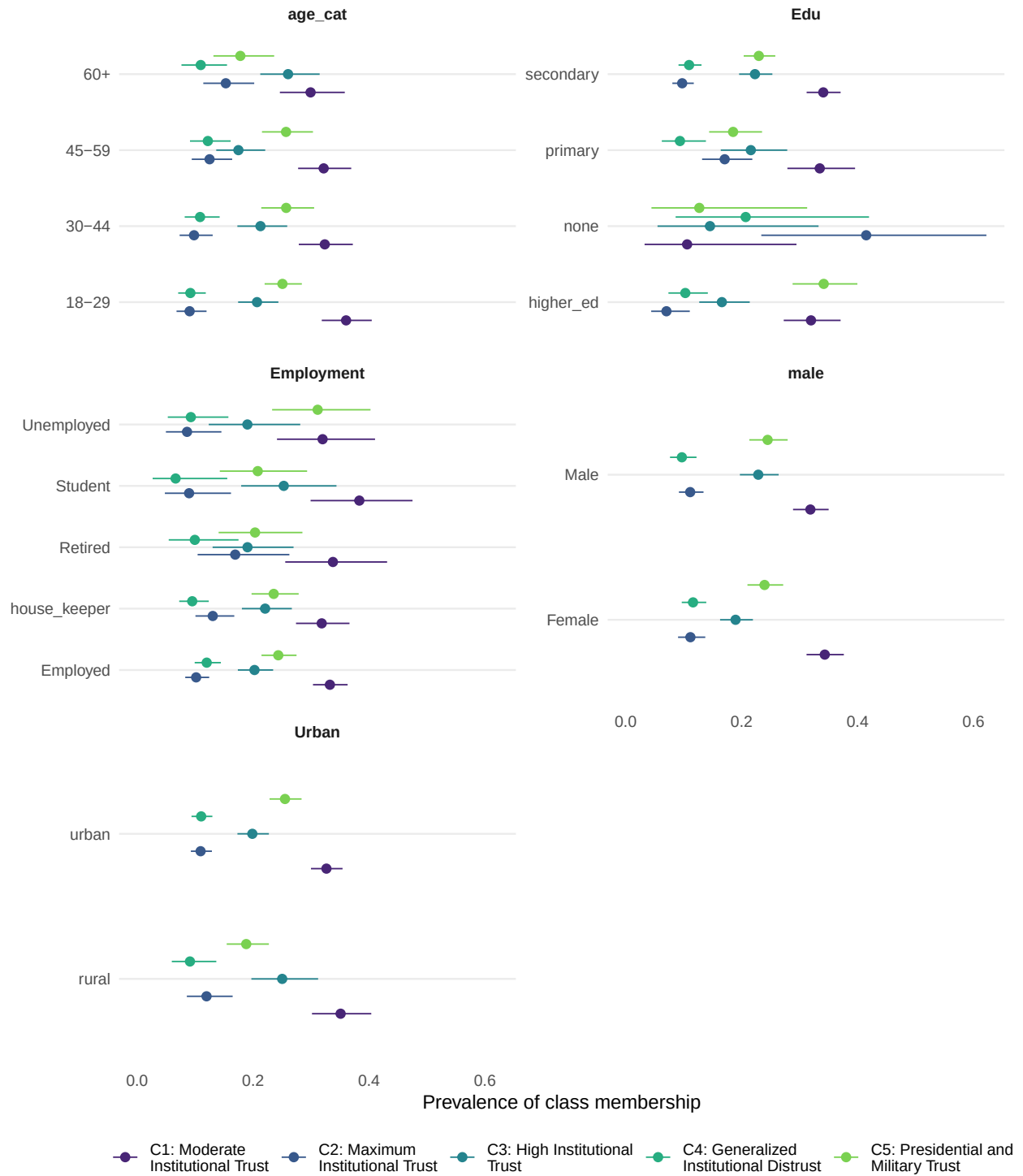


Figure 5: Design-based prevalence of each latent class within each demographic level, one panel per demographic, with 95% intervals from the JK_n replicate weights. Within a level the prevalences sum to one across classes. Wide bands reflect few PSUs in that group, not a coding artifact.

16 Exporting results

Why this section matters: it makes the analysis usable beyond the render. The enumeration, the design-based interval table, and the fully scored respondent file are the artifacts that feed manuscripts, slides, and downstream merges, and writing them out is what turns a one-time report into a reproducible dataset others can build on.

```
readr::write_csv(enum, file.path(cfg$out_dir, "lca_enumeration.csv"))
readr::write_csv(ci_tbl |>
  mutate(across(where(is.numeric), ~ round(.x, 5)),
    class_label = class_labels$Label[class]),
  file.path(cfg$out_dir, "lca_measurement_model_with_CIs.csv"))
```

Two files are written to `cfg$out_dir` here: the enumeration table and the full measurement model with design-based confidence intervals (one row per probability, with `item`, `category`, `class`, and the applied `class_label` as columns). The class-label sheet is `llm_labels.csv` (written by the naming section), and the case-level product is the labelled SPSS file from the membership section.

17 Adapting this script to your survey: a checklist

1. **Edit the two data chunks.** They are ordinary dplyr you own: chunk one reads the .sav, recodes nonresponse to NA, recodes demographics to character, flags the analysis cases (`keep`), and strips nonresponse value labels; chunk two builds the dictionary and the retained-cases plot. Everything after them is machinery.
2. **Items and cases.** Items may have **different numbers of categories** and different response labels; both are read per item from the data. The analysis uses complete cases only (the `keep` flag); cases with any missing item are never assigned a class, honestly, and appear as NA in the SPSS handoff.
3. **Set `cfg$strata`, `cfg$psu`, `cfg$weight`** to your design columns. This is the analogue of `svyset psu [pweight = w], strata(stratum)`. If your file ships **replicate weights** instead of strata and PSU identifiers, replace each `svydesign(...)` plus `as.svrepdesign(...)` pair with a single `svrepdesign(repweights = ..., weights = ..., type = ...)`; the rest of the variance code is unchanged.
4. **Set `cfg$aux`** to the demographics you want in the profile. Demographics used to build the weights (age, for instance) are fine and often helpful here; the only variables to exclude are the survey items that defined the classes.
5. **Choose K by iterating.** With `cfg$K_force = NULL` only the enumeration runs; read BIC, entropy, and the inclusion probabilities, set a candidate K, re-render, and judge that model's discrimination, bivariate residuals, and profiles. Nothing is auto-selected.
6. **Trust checks.** The validation section should show the EM matching `poLCA` at unit weights before you trust the weighted output.
7. **Mind the degrees of freedom.** Your design-based intervals carry roughly (number of PSUs minus number of strata) degrees of freedom; with few PSUs they will be wide, and that width is honest.

8. **Read the demographic profile as design-based prevalences (the profiling section).** It reports each class's prevalence within each demographic level with design-based intervals from the JKn replicate weights; a prevalence within a stratifying variable's levels is a well-defined domain estimate. Wide bands reflect few PSUs in a group, not an error. These estimates treat the measurement model as given and do not propagate the step-one latent class uncertainty.
9. **Keep `survey_lca_source.R` next to this document.** It holds the plotting and data-prep helpers, the weighted EM engine, and the LLM class-labeling functions, and is sourced in the setup chunk. The functions are dataset-agnostic, so the same file serves any study, but the render fails if it is not in the working directory.
10. **The handoff.** Once `cfg$K_force` is set and `llm_labels.csv` exists, the membership section writes `survey_with_classes.sav`: the preserved original frame plus a labelled `class` variable (assigned for complete cases, NA otherwise), ready for analysts.
11. **Class labels.** One rule: when `outputs/llm_labels.csv` exists it is used; otherwise the LLM drafts it once (one call per class) and writes it. Edit that file to take over naming; delete it to redraft. At work set `cfg$compass_base_url` and `cfg$llm_model`; ellmer reads `OPENAI_API_KEY` from `.Renvirom` on both machines, and no key-handling code exists.

```
future::plan(future::sequential) # close any parallel workers
```